

瞭解 GKE NodeLocal DNSCache

來源：<https://codelabs.developers.google.com/codelabs/nodelocal-dns-cache-gke?hl=zh-tw>

步驟數：8

建立自訂 VPC 和標準 GKE 叢集，探索預設的 NodeLocal DNSCache 功能。

目錄

1. [1. 總覽](#)
2. [2. 設定實驗室](#)
3. [3. 設定環境](#)
4. [4. 建立網路位址轉譯 \(NAT\) 閘道，用於網際網路通訊](#)
5. [5. 部署 GKE 叢集並驗證](#)
6. [6. 停用 NodeLocal DNSCache 並驗證](#)
7. [7. 清除所用資源](#)
8. [8. 後續步驟/瞭解詳情](#)

1. 總覽

DNS 快取會先將 Pod DNS 要求傳送至相同節點上的本機快取，藉此縮短 DNS 查詢延遲時間。這可讓 DNS 查詢時間更一致，並減少對 kube-dns 或 Cloud DNS 的 DNS 查詢次數。

在本實驗室中，您將測試 **NodeLocal DNSCache** 如何處理 GKE 叢集中的 DNS 流量。您將驗證執行 1.34.1-gke.3720000 以上版本的 GKE Standard 叢集，確認這項功能是否預設啟用。然後停用，看看關閉這項功能後設定會如何變更。

注意：GKE Autopilot 叢集預設也會啟用 **NodeLocal DNSCache**，但無法關閉。

目標

在本實驗室中，您將瞭解如何執行下列工作：

- 建立自訂虛擬私有雲、子網路和防火牆規則
 - 部署發布管道為「快速」的標準 GKE Standard 叢集
 - 執行測試，確認已啟用 LocalNode DNS 快取
 - 停用快取，並驗證沒有快取的狀態
-

步驟 2 · 約 0 分鐘

2. 設定實驗室

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud console interface. At the top, there is a 'Select a project' dropdown menu. Below it, the 'Select a project' page is visible, featuring a search bar labeled 'Search projects and folders'. To the right of the search bar is a 'NEW PROJECT' button. Below the search bar, the 'New Project' form is shown. It includes a 'Project name' field with the text 'My Project 13475', a 'Project ID' field with the text 'inbound-analogy-389614' (highlighted with a blue box), and a 'Location' field. There are also buttons for 'CREATE' and 'CANCEL' at the bottom.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Google Cloud Shell interface. The browser's address bar displays 'qwiklabs-gcp-b3de7b9575cb8d19'. The terminal window has a dark background and shows the following text: 'Welcome to Cloud Shell! Type "help" to get started.' followed by the prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' with a red cursor.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

3. 設定環境

請注意，啟用部分資源會產生費用。請務必先瞭解這點再執行。為降低費用，請務必在完成實驗室後刪除資源。建議您先查看帳單，確認是否能接受這筆費用。💡

我們將建立含有防火牆規則的自訂虛擬私有雲。如果您已有 VPC 和專案，可以略過這個部分。

開啟控制台右上方的 Cloud Shell。然後依照下列方式設定：

Search (/) for resources, docs, products and more

Search



1. 啟用本實驗室中會用到的一些 API

```
gcloud services enable dns.googleapis.com
gcloud services enable servicedirectory.googleapis.com
gcloud services enable container.googleapis.com
```

2. 設定一些變數。這些變數是專案 ID 和虛擬私有雲名稱 (您會在步驟 3 建立虛擬私有雲)。

```
export projectid=$(gcloud config get-value project)
export vpc_name=gke-cache-vpc
export subnet_name=mainsubnet
export region=us-east1
export zone=us-east1-b
export cluster_name=cache-gke-cluster
export channel=rapid

export machine_type=e2-standard-4
echo $projectid
echo $vpc_name
```

3. 現在建立名為 `gke-cache-vpc` 的自訂 VPC

```
gcloud compute networks create $vpc_name --subnet-mode=custom --project=$projectid \
--subnet-mode=custom \
--mtu=1460 \
--bgp-routing-mode=global
```

4. 在新虛擬私有雲中建立子網路

```
gcloud compute networks subnets create $subnet_name \
--network=$vpc_name \
--range=10.0.88.0/24 \
--region=$region \
--enable-private-ip-google-access \
--project=$projectid
```

5. 在虛擬私有雲中新增防火牆規則

```
gcloud compute firewall-rules create $vpc_name-allow-internal \  
  --network=$vpc_name --allow=tcp,udp,icmp --source-ranges=10.0.88.0/24  
  
gcloud compute firewall-rules create $vpc_name-allow-ssh \  
  --network=$vpc_name --allow=tcp:22 --source-ranges=35.235.240.0/20
```

步驟 4 · 約 0 分鐘

4. 建立網路位址轉譯 (NAT) 閘道，用於網際網路通訊

我們需要授予連出網際網路的外部存取權，因此請建立並附加 Cloud NAT 閘道。

在 **Cloud Shell** 中使用下列指令

1. 建立 Cloud NAT 和 NAT 閘道

```
gcloud compute routers create gke-nat-router --network=$vpc_name --region=$region
```

```
gcloud compute routers nats create gke-nat-gw \  
  --router=gke-nat-router --region=$region \  
  --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges
```

5. 部署 GKE 叢集並驗證

1. 在 Google Cloud Shell 中建立名為 `cache-gke-cluster` 的叢集。如果 GKE Standard 叢集執行 `1.34.1-gke.3720000` 以上版本，預設會啟用 NodeLocal DNSCache。(佈建叢集需要 4 到 10 分鐘的時間)

```
gcloud container clusters create $cluster_name \
--zone=$zone --network=$vpc_name --subnetwork=$subnet_name \
--release-channel=$channel --machine-type=$machine_type \
--enable-ip-alias
```

2. 叢集建立完成後，請連線：

```
gcloud container clusters get-credentials $cluster_name --zone $zone
```

3. 現在讓我們確認 NodeLocal DNSCache 是否已啟用。

這些指令會確認版本為 `1.34.1-gke.3720000` 以上，並確認本機代理程式正在執行，且服務

```
kubectl version | grep "Server Version"

kubectl get pods -n kube-system -o wide | grep node-local-dns -w

kubectl get svc,endpoints -n kube-system -l k8s-app=kube-dns
```

4. 接著執行下列指令 (這會在主機網路上建立具備特殊權限的 Pod，以驗證節點的 iptables 規則是否主動攔截 DNS 流量，並將其轉送至本機快取)

```
export KUBEDNS_IP=$(kubectl get svc kube-dns -n kube-system -o jsonpath='{.spec.clusterIP}')

kubectl run node-inspector --quiet --rm -it --image=alpine --privileged --restart=Never \
--overrides='{ "spec": { "hostNetwork": true } }' -- \
sh -c "apk add --no-cache iptables && iptables-save | grep -E '169.254.20.10|$KUBEDNS_IP'"
```

應尋找的項目：尋找 `-j NOTRACK`。這會確認 DNS 流量是否略過連線追蹤表。

```
6 iptables-save | grep -E '169.254.20.10|$KUBEDNS_IP'
-A PREROUTING -d 169.254.20.10/32 -p tcp -m tcp --dport 53 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A OUTPUT -s 34.118.224.10/32 -p tcp -m tcp --sport 8080 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A OUTPUT -d 34.118.224.10/32 -p tcp -m tcp --dport 8080 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A OUTPUT -d 34.118.224.10/32 -p udp -m udp --dport 53 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A OUTPUT -d 34.118.224.10/32 -p tcp -m tcp --dport 53 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A OUTPUT -s 34.118.224.10/32 -p udp -m udp --sport 53 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A OUTPUT -s 34.118.224.10/32 -p tcp -m tcp --sport 53 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A OUTPUT -s 169.254.20.10/32 -p tcp -m tcp --sport 8080 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A OUTPUT -d 169.254.20.10/32 -p tcp -m tcp --dport 8080 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A OUTPUT -d 169.254.20.10/32 -p udp -m udp --dport 53 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A OUTPUT -d 169.254.20.10/32 -p tcp -m tcp --dport 53 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A OUTPUT -s 169.254.20.10/32 -p udp -m udp --sport 53 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A OUTPUT -s 169.254.20.10/32 -p tcp -m tcp --sport 53 -m comment --comment "NodeLocal DNS Cache: skip conntrack" -j NOTRACK
-A INPUT -d 34.118.224.10/32 -p udp -m udp --dport 53 -m comment --comment "NodeLocal DNS Cache: allow DNS traffic" -j ACCEPT
-A INPUT -d 34.118.224.10/32 -p tcp -m tcp --dport 53 -m comment --comment "NodeLocal DNS Cache: allow DNS traffic" -j ACCEPT
-A INPUT -d 169.254.20.10/32 -p udp -m udp --dport 53 -m comment --comment "NodeLocal DNS Cache: allow DNS traffic" -j ACCEPT
-A INPUT -d 169.254.20.10/32 -p tcp -m tcp --dport 53 -m comment --comment "NodeLocal DNS Cache: allow DNS traffic" -j ACCEPT
-A OUTPUT -s 34.118.224.10/32 -p udp -m udp --sport 53 -m comment --comment "NodeLocal DNS Cache: allow DNS traffic" -j ACCEPT
-A OUTPUT -s 34.118.224.10/32 -p tcp -m tcp --sport 53 -m comment --comment "NodeLocal DNS Cache: allow DNS traffic" -j ACCEPT
-A OUTPUT -s 169.254.20.10/32 -p udp -m udp --sport 53 -m comment --comment "NodeLocal DNS Cache: allow DNS traffic" -j ACCEPT
-A OUTPUT -s 169.254.20.10/32 -p tcp -m tcp --sport 53 -m comment --comment "NodeLocal DNS Cache: allow DNS traffic" -j ACCEPT
```

步驟 6 · 約 0 分鐘

6. 停用 NodeLocal DNSCache 並驗證

我們現在移除最佳化功能，看看沒有這項功能會發生什麼事。

1. 前往 Cloud Shell 並執行下列指令。**注意：**這會觸發節點重建作業，由於 GKE 會循環執行個體，因此每個節點集區通常需要 **3 到 5 分鐘**

```
gcloud container clusters update $cluster_name --zone=$zone --update-addons=NodeLocalDNS=DISABLED --quiet

kubectl get pods -n kube-system -o wide | grep node-local-dns -w
```

您應該不會在精靈集看到任何這些 Pod，因為它們已移除。

2. 重新執行測試

```
kubectl run node-inspector --quiet --rm -it --image=alpine --privileged --restart=Never \
  --overrides='{ "spec": { "hostNetwork": true } }' -- \
  sh -c "apk add --no-cache iptables && iptables-save | grep -E '169.254.20.10|$KUBEDNS_IP'"
```

停用外掛程式後，輸出內容就不會再包含 `-j NOTRACK` 規則，也不會提及 169.254.20.10 IP 位址。這表示您無法再享有本機快取的好處

```
(1/4) Installing libmnl (1.0.5-r2)
(2/4) Installing libnftnl (1.3.0-r0)
(3/4) Installing libxtables (1.8.11-r1)
(4/4) Installing iptables (1.8.11-r1)
Executing busybox-1.37.0-r30.trigger
OK: 10.1 MiB in 20 packages
-A KUBE-SERVICES -d 34.118.224.10/32 -p udp -m comment --comment "kube-system/kube-dns:dns cluster IP" -m udp --dport 53 -j KUBE-SVC-TCOU7JCQXEZGVUNU
-A KUBE-SERVICES -d 34.118.224.10/32 -p tcp -m comment --comment "kube-system/kube-dns:dns-tcp cluster IP" -m tcp --dport 53 -j KUBE-SVC-ERIFXISQEP7F7OF4
-A KUBE-SVC-ERIFXISQEP7F7OF4 ! -s 10.168.2.0/24 -d 34.118.224.10/32 -p tcp -m comment --comment "kube-system/kube-dns:dns-tcp cluster IP" -m tcp --dport 53 -j KUBE-MARK-MASQ
-A KUBE-SVC-TCOU7JCQXEZGVUNU ! -s 10.168.2.0/24 -d 34.118.224.10/32 -p udp -m comment --comment "kube-system/kube-dns:dns cluster IP" -m udp --dport 53 -j KUBE-MARK-MASQ
```

7. 清除所用資源

```
# 1. Delete the GKE Cluster
# This will remove the node and all system pods (including kube-dns)
gcloud container clusters delete $cluster_name --zone=$zone --quiet

# 2. Delete the Cloud NAT and Router
# It is best practice to remove these before the VPC
gcloud compute routers nats delete gke-nat-gw --router=gke-nat-router --region=$region --quiet
gcloud compute routers delete gke-nat-router --region=$region --quiet

# 3. Delete the Firewall Rules
gcloud compute firewall-rules delete $vpc_name-allow-internal $vpc_name-allow-ssh --quiet

# 4. Delete the Subnet and VPC
gcloud compute networks subnets delete $subnet_name --region=$region --quiet
gcloud compute networks delete $vpc_name --quiet
```

步驟 8 · 約 0 分鐘

8. 後續步驟/瞭解詳情

您可以參閱 [GKE 網路說明文件和應用實例](#)

Codelab：透過 [Private Service Connect](#) 端點，使用 [Python SDK](#) 存取 [Gemini 3 Pro](#) 對話

Codelab：使用 [ADK](#) 建構 AI 代理：基礎知識

挑戰下一個實驗室

繼續完成 Google Cloud 任務，或查看其他 Google Cloud Skills Boost 實驗室：

- [設定 Google Kubernetes Engine 網路](#)
 - [探索生成式 AI - Vertex AI](#)
 - [Google Cloud 上的代理式 AI](#)
-